

4.1 Operadores Aritméticos e Identidad Numérica

Dart proporciona operadores aritméticos estándar sobre tipos numéricos (`num`, `int`, `double`). A nivel de compilación, estos operadores están mapeados directamente a métodos en las clases numéricas primarias.

- **Suma (+) y Resta (-):** Operaciones básicas entre números o concatenación de cadenas en el caso del operador `+`.
- **Multiplicación (*):** Multiplica dos operandos.
- **División Tradicional (/):** Realiza una división decimal. **Nota importante:** En Dart, la división mediante `/` siempre retorna una variable de tipo `double`, incluso si los dos operandos son de tipo `int` (por ejemplo, `4 / 2` produce `2.0`).
- **División Entera Truncada (~/):** Realiza la división y retorna un resultado de tipo `int` descartando la parte decimal sin redondear.
- **Módulo (%):** Retorna el residuo de la división entera entre dos números.

4.2 Operadores Relacionales y Verificación de Tipos

Los operadores relacionales permiten evaluar condiciones devolviendo siempre un valor booleano estricto (`true` o `false`).

- **Comparaciones Estándar:** `>` (mayor que), `<` (menor que), `>=` (mayor o igual), `<=` (menor o igual).
- **Igualdad y Desigualdad:** `==` (igual a) y `!=` (diferente de).

Mecánica interna: El operador `==` evalúa la igualdad estructural llamando al método `operator ==` del objeto. Para verificar si dos referencias apuntan exactamente a la misma instancia en memoria, se utiliza la función `identical(a, b)`.

Operadores de Test de Tipos

- `is`: Retorna `true` si el objeto es del tipo especificado.
- `is!`: Retorna `true` si el objeto **no** es del tipo especificado.

4.3 Operadores de Incremento y Decremento: Prefijo vs. Posfijo

Los operadores `++` y `--` modifican el valor de una variable numérica sumando o restando `1`. La posición del operador respecto a la variable cambia drásticamente el orden de evaluación en las expresiones:

Posfijo (`variable++` / `variable--`)

Evalúa la expresión utilizando el valor **actual** de la variable y, posteriormente, realiza el incremento o decremento en memoria.

```
int a = 5;
int b = a++; // b recibe el valor 5, luego 'a' se incrementa a 6.
```

Prefijo (++variable / --variable)

Incrementa o decrementa el valor de la variable en memoria **antes** de evaluar la expresión.

```
int x = 5;
int y = ++x; // 'x' se incrementa a 6, luego 'y' recibe el valor 6.
```

4.4 Operadores Lógicos y Evaluación en Cortocircuito

Permiten combinar o negar expresiones booleanas para tomar decisiones en el flujo de ejecución.

- **&&** (AND Lógico): Retorna **true** únicamente si ambas expresiones son verdaderas.
- **||** (OR Lógico): Retorna **true** si al menos una de las expresiones es verdadera.
- **!** (NOT Lógico / Negación): Invierte el valor booleano (**!true** es **false**).

Evaluación en Cortocircuito (Short-Circuit Evaluation)

Los operadores **&&** y **||** optimizan la ejecución de la CPU deteniendo la evaluación en cuanto el resultado global está determinado:

- En **A && B**: Si **A** es **false**, la expresión **B** **nunca se ejecuta** porque el resultado global inevitablemente será **false**.
- En **A || B**: Si **A** es **true**, la expresión **B** **nunca se ejecuta** porque el resultado global inevitablemente será **true**.

4.5 Ejemplo

Creen el archivo **bin/d4_operadores.dart** en VS Code. El siguiente programa simula un motor de reglas para la aprobación de créditos académicos y materias en ESCOM evaluando operadores aritméticos, relacionales, de incremento y lógicos.

```
/// Módulo D4: Operadores Aritméticos, Relacionales, Incremento/Decremento y
Lógicos
/// Demostración práctica de evaluación matemática y lógica en Dart.

void main() {
  print('=====');
  print('    EVALUADOR DE REGLAS DE NEGOCIO Y OPERADORES - ESCOM    ');
  print('=====');

  // 1. Operadores Aritméticos y División Entera
  final int creditosTotalesPlan = 380;
  final int creditosAprobados = 285;

  final double porcentajeAvance = (creditosAprobados / creditosTotalesPlan) * 100;
  // División decimal
```

```
final int semestresEquivalentes = creditosAprobados ~/ 45;
// División entera
final int creditosSobrantes = creditosAprobados % 45;
// Módulo

print('\n--- Avance Académico ---');
print('Porcentaje de avance: ${porcentajeAvance.toStringAsFixed(2)}%');
print('Semestres equivalentes cursados: $semestresEquivalentes');
print('Créditos remanentes del módulo: $creditosSobrantes');

// 2. Operadores de Incremento y Decremento (Prefijo vs Posfijo)
print('\n--- Evaluación de Incremento y Decremento ---');
int contadorIntentosExamen = 1;

print('Valor inicial del contador: $contadorIntentosExamen');
print('Uso de posfijo (contador++): ${contadorIntentosExamen++}'); // Imprime 1,
// luego vale 2
print('Valor tras posfijo: $contadorIntentosExamen');
print('Uso de prefijo (++contador): ${++contadorIntentosExamen}'); // Suma a 3 y
// luego imprime 3

// 3. Operadores Relacionales y de Tipo (is / is!)
final dynamic promedioGeneral = 8.5;
final bool tieneRegularidad = true;

print('\n--- Validación de Tipos y Comparaciones ---');
print('¿El promedio es numérico? ${promedioGeneral is num}');
print('¿El promedio es texto? ${promedioGeneral is! String}');

// 4. Operadores Lógicos y Cortocircuito
print('\n--- Motor de Reglas de Selección de Dictamen ---');

final int materiasReprobadas = 0;
final int horasServicioSocial = 480;

// Evaluación lógica compuesta:
// Se requiere ser regular, tener promedio mayor o igual a 8.0,
// no tener reprobadas Y haber concluido el servicio social.
final bool elegibleTitulacionDirecta = tieneRegularidad &&
    (promedioGeneral >= 8.0) &&
    (materiasReprobadas == 0) &&
    (horasServicioSocial >= 480);

print('¿Es elegible para Titulación Directa?: $elegibleTitulacionDirecta');

// Demostración de Cortocircuito
print('\n--- Demostración de Evaluación en Cortocircuito ---');
bool funcionCostosa() {
    print(' [ADVERTENCIA] Se ejecutó la función de verificación en BD. ');
    return true;
}

// En una condición OR, si el primer término es true, la función costosa NO debe
// ejecutarse.
```

```

print('Evaluando: true || funcionCostosa()...');
if (true || funcionCostosa()) {
    print(' -> Resultado: Condición cumplida sin evaluar la segunda parte.');
```

// En una condición AND, si el primer término es false, la función costosa NO debe ejecutarse.

```

print('Evaluando: false && funcionCostosa()...');
if (false && funcionCostosa()) {
    // No entra
} else {
    print(' -> Resultado: Condición rechazada directamente sin evaluar la segunda parte.');
```

```

}

print('=====');
```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d4_operadores.dart
```

4.6 Ejercicios Prácticos

Implementen los siguientes dos programas en su entorno local para consolidar la manipulación de operadores aritméticos, lógicos y el control del orden de evaluación.

Ejercicio 1: Calculadora de Tarifas y Descuentos con Reglas Lógicas

Crea un script en Dart que determine el costo final de un taller de titulación en ESCOM considerando las siguientes reglas:

1. Define las variables base: `double costoBase = 1500.0`, `bool esAlumnoEscom`, `bool esEgresado`, `double promedio`.
2. Asigna valores de prueba a las variables.
3. Aplica un descuento según las siguientes condiciones lógicas:
 - Si es alumno activo de ESCOM **Y** tiene un promedio mayor o igual a `9.0`, obtiene un `30%` de descuento.
 - Si es egresado **O** alumno de ESCOM con promedio entre `8.0` y `8.9`, obtiene un `15%` de descuento.
 - En cualquier otro caso, paga el costo total (`0%` de descuento).
4. Muestra en consola el costo original, el porcentaje de descuento aplicado y el monto final a pagar.

Ejercicio 2: Rastreador de Intentos de Autenticación (Prefijo/Posfijo)

Escribe un programa que simule el bloqueo de acceso a una plataforma tras una serie de intentos fallidos de inicio de sesión:

1. Define la variable `int intentosRealizados = 0` y `const int maxIntentos = 3`.
 2. Simula una serie de 4 intentos fallidos dentro de un bucle o secuencia de instrucciones.
 3. En cada intento, incrementa la variable usando el operador de incremento en la modalidad **posfijo** dentro de un mensaje impreso, analizando el comportamiento de la variable en pantalla.
 4. Evalúa con un operador relacional si `intentosRealizados >= maxIntentos` e imprime si la cuenta ha sido bloqueada.
-